

A Fragility Probe for Carved Structure

A proposed bit-depth sweep for testing whether precision demand predicts quantization sensitivity.

Companion protocol note to The River and the Canyon and What Kind of Water Carves the Mountain?

Status: v0.1 proposed experiment — not performed. This document does not prove the two-axis framework from *What Kind of Water Carves the Mountain?* It specifies the smallest clean experiment that would test **one** of its predictions, states what would support that prediction, and states what would weaken it. It is written so that someone with the compute, tooling, and evaluation infrastructure — which the author does not currently have — could pick it up and run it. The author's contribution here is the question and the design, not the data. **Pre-register the stress profile, matched task pairs, scoring rules, negative controls, scorer-sensitivity checks, and decision criteria before running** — if any of these are chosen after results are visible, the method weakens to post-hoc tuning.

The claim under test

The companion paper proposes two distinct axes: *provenance* (what kind of structure the training medium carved) and *fragility* (how much precision that structure needs to survive coarsening). This protocol tests only the fragility axis, and only its cleanest prediction:

At a fixed bit depth, capabilities that demand exact, precision-sensitive computation should retain a smaller fraction of their full-precision performance than capabilities that are broad and lower in precision demand — when the two are matched on everything except precision demand.

A note on what a positive result would and would not show. Even after the length, decoding, and generic-sensitivity controls below are applied, a confirmed gap is **candidate capability-level stress sensitivity**, not proof of a mechanism called “precision demand.” “Precision demand” names the *hypothesis variable* used to construct and label the pairs; it is not a claim about an internal cause the behavioral measurement can see. The honest reading of any signal is “this task family loses more retention under this stress, after generic length/decoding/perturbation artifacts are subtracted” — which is a behavioral result, with mechanism left to separate causal evidence (see the boundary diagram: the method does not localize cause).

This is deliberately narrow. It does not test provenance. It does not establish margin geometry. It tests whether “fragility” shows up as a *measurable difference in degradation curves* inside a single model, rather than remaining an aesthetic category.

The practical gap it targets: peak FP16/BF16 accuracy shows that a behavior is *present* under favorable numerical conditions; it does not by itself show whether that behavior remains *stable*

under downstream compression or deployment stress. Whether stress-retention actually carries information that peak accuracy misses is the open question — this design tests it; it does not assume it.

Why this experiment first, and not the provenance comparison

The conceptually interesting question — does a code-trained model degrade differently from a text-trained one? — has too many confounders to be a clean first move: model choice, training mixture, benchmark quality, calibration data, tokenizer effects, and the ever-present risk of measuring task difficulty while believing you are measuring provenance.

The within-model bit-depth sweep removes almost all of that. One model, one quantization recipe, one evaluation set, held constant. The only thing that varies between the paired tasks is precision demand. Establish the baseline fragility signature here first; the provenance comparison is only interpretable once you know the within-model effect is real.

Design

One model. A single dense open-weights model. (Mixture-of-experts is deferred — router behavior under quantization is a separate failure mode and adds a variable.)

One quantization recipe, fixed and fully specified — including whether it quantizes weights only or weights and activations, since the two behave differently and a reader needs to know which is being tested. Calibration data is **held fixed across all bit depths**. Changing the calibration set between rungs introduces a variable that masquerades as a precision effect. Make this checkable rather than promised: use one calibration file, record its hash, and reuse the identical file at every width — so “we held calibration constant” is something a reader can verify, not something they have to trust. If task pairs span text, code, and math, a mixed-domain calibration set is preferable to pure text, which could bias quantization toward language behavior and let a critic attribute code degradation to calibration mismatch rather than precision demand.

A bit-depth ladder: FP16/BF16 → INT8 → INT4. Optionally INT6 between, and INT3 if you want to see the cliff. The full-precision run is the per-task baseline.

Matched task pairs. This is the real experiment, and pair construction is where it succeeds or fails. Each pair shares source material, length band, and prompt skeleton; only the *instruction* changes, so the two halves differ in precision demand and as little else as possible:

Domain	Broad / lower-precision-demand half	Narrow / precision-demanding half
Text	Broad summarization	Exact logical / temporal tracking
Code	High-level explanation of what code does	Exact output on hidden input
Math	Conceptual explanation	Multi-step numerical execution
State	Loose story comprehension	Exact entity / state tracking

The split is by precision demand, not apparent complexity. This is an explicit design rule, because the intuition “hard task = narrow, easy task = broad” is wrong and will misclassify pairs. The narrow half is the one that requires *exact state preservation, step execution, variable binding, numerical or symbolic precision, or fragile tool/format constraints* — where a small deviation flips correctness. A task can be conceptually complex but robust (many acceptable phrasings, graceful degradation), and another can look simple but be brittle (one exact answer, cliff-like failure). Classify each half by *what kind of correctness it demands*, not by how difficult it appears.

Pre-register the precision-demand labels. Before running the model, assign each candidate task its broad/narrow label using a declared rubric — output-space size, tolerance window, state/binding demand, error amplification, format strictness, context/length sensitivity — and publish the completed labels with the task set. If multiple people construct pairs, resolve label disagreements before any model results are visible. This closes the most dangerous post-hoc move: seeing which tasks degraded and concluding *those must have been the narrow ones*. The label must precede the result, or the experiment is circular.

Concrete example (text pair, same source article): broad / lower-precision-demand — “list the three required facts from the article” (a checklist score, many acceptable phrasings); narrow / precision-demanding — “determine whether event A occurred before event B, and identify which entity changed state” (exact ordering and binding; one small slip flips it). Same article, comparable scoring strictness, only precision demand differs — this is what a clean pair looks like, and why “broad” here does not mean “easy.”

Four pairs is a pilot. The point of pairing is to isolate precision demand instead of accidentally measuring that one task was simply harder to begin with. To keep that honest, hold each pair to explicit acceptance criteria before the run: both halves drawn from the same source item where possible, output budgets within roughly 10% of each other (so one side isn’t penalized for length), comparable FP16 baselines, and no tasks where a trivial substring of the prompt is a correct answer. Record why any candidate pair was excluded. A pair that can’t meet these isn’t a clean test of precision demand and shouldn’t be in the pilot.

Context-depth alignment. The evidence required to solve each half of a pair should sit at comparable relative positions in the input context. If the narrow half depends on a late-context entity, state change, or calculation, the broad half should depend on comparably late evidence. Otherwise the measured gap may reflect positional degradation, KV-cache effects, or long-context compounding rather than precision demand — a model can lose late-context binding under stress for reasons that have nothing to do with how exact the required answer is. Matching evidence position closes that confound.

Match generation length and state-maintenance load, not just task difficulty. A specific confound applies to compositional or chained pairs: maintaining intermediate state across a long generation window drives up activation magnitudes (salient-channel outliers), and outliers are a primary cause of quantization damage independent of any “compositional” content. So a chained task could show a larger retention drop than its control purely because it stresses the residual stream harder — an activation-outlier effect (the numerical-plumbing mechanism), not a structural-composition effect. To keep the compositional comparison clean, the non-compositional control must be matched on **generation length and state-**

maintenance demand, not difficulty alone — so that both arms place comparable load on the residual stream and any remaining gap is attributable to the handoff rather than to outlier blowout. Exact equality of token counts is neither realistic nor required: match within a pre-registered tolerance and **record the residual length and step-count mismatch as a covariate** (or stratification variable) so it can be checked against the effect, rather than pretending it is zero. Without this, the matched-pair instrument cannot distinguish a composition break from a layer-norm clipping artifact.

Pre-register the tolerances, don't hard-code universal numbers. Each of these guards — baseline matching, negative-control acceptance, context-depth alignment, scorer-sensitivity — needs a numeric tolerance, but the right value is not known before a pilot, and inventing universal constants (a fixed ΔR ceiling for controls, a fixed baseline-gap limit, a fixed sentence-window) just creates an arbitrary target for criticism. Instead: declare the tolerances for this run *before seeing results*, and if a control is violated and the violation could plausibly explain the apparent gap, classify the run as Outcome B or C rather than reporting it as support. The discipline is in fixing the bar before the run, not in any particular number.

Negative controls. Alongside the four contrast pairs, include at least one *broad–broad* pair and one *narrow–narrow* pair — two tasks of the same precision class, scored with identical strictness. These should *not* show a within-pair retention gap. If they do, the apparent fragility effect in the contrast pairs is suspect: the paired design itself may be leaking task difficulty or metric bias rather than precision demand. The controls are how the experiment checks its own instrument before trusting its main result.

Stress specification: format and calibration are part of the instrument

“Quantization” is not one stress, and the protocol must not treat it as one. The most comprehensive study to date (Kurtic et al., “Give Me BF16 or Give Me Death,” arXiv 2411.02355, ~500,000 evaluations on Llama-3.1) found the spread across formats is large and non-obvious: **FP8 (W8A8-FP) effectively lossless across scales, well-tuned INT8 (W8A8-INT) only 1–3% degradation, and INT4 weight-only (W4A16-INT) more competitive than expected.** So “INT4 failed” is uninterpretable on its own — INT4 *which method*, quantizing *what*, with *which calibration*, on *which task*. A retention number without its stress specification is not a result.

Every run must therefore declare its full stress profile, so a reader can tell what was actually tested and reproduce it:

1. **Quantization format and method** (e.g. FP8 W8A8, INT8 W8A8, INT4 W4A16; GPTQ / AWQ / SmoothQuant / etc.).
2. **What is quantized** — weights only, or weights *and* activations (the two behave differently).
3. **Bit-depth ladder** used (FP16/BF16 → ... → lowest width).
4. **Calibration file hash** — so “calibration held constant” is verifiable, not promised.
5. **Calibration distribution summary** — domain mix (text/code/math proportions), token and length distribution, language, and whether the calibration set resembles the evaluated tasks. Calibration distribution measurably affects PTQ behavior and worst-

case reliability; fixing the hash ensures *repeatability*, describing the distribution ensures *interpretability*.

Cross-calibration invariance is a required validation gate, not an optional check. PTQ methods compute their clipping thresholds and scaling factors *from* the calibration set, so a skewed calibration file distorts which pathways survive coarsening — which means a retention ranking could be measuring the cross-entropy distance between each task and the calibration data, not the model’s intrinsic precision demand. This is a genuine way to get a clean-looking result that is purely a calibration artifact. To close it: run Tier 0 across **two distinct calibration hashes** (e.g. a code-heavy set and a prose-heavy set). The fragility signature is validated **only if the relative retention ranking of the task pairs is invariant across both calibration manifolds**. If the ranking flips with calibration, the result is discarded as an optimization artifact, not reported as fragility. A single-calibration result is a provisional signal at best; invariance across two is the minimum bar for treating the ranking as a property of the model rather than of the setup.

This turns “quantization” from a vague hammer into a defined stress profile, and it is the difference between a result a reviewer can interpret and one they can wave away.

Reliability axes deferred from Tier 0 scoring

These dimensions are **not part of the baseline Tier 0 decision rule** — folding them into Tier 0 would turn a runnable pilot into a heavy instrumentation project, and the first question (does precision demand predict a retention gap under comparable scoring) can be answered without them. They are deferred from *Tier 0 scoring*, not abandoned: some are **eligible as Tier 1 diagnostics if Tier 0 returns Outcome A** (uncertainty/calibration retention is exactly track 1b below; cross-stress is track 1c). They are recorded here so the richer stress-space is on the map and so a reader sees these are *intentional deferrals to keep Tier 0 clean*, not forgotten confounds.

- **Uncertainty / calibration retention.** Compression can decouple *accuracy* from *uncertainty* — a model may stay correct but become poorly calibrated, or answer wrongly with high confidence (work on compression-and-uncertainty argues accuracy-only evaluation is insufficient for deployment readiness). This fits the silent-failure theme exactly: the better questions are eventually not only “did it stay correct?” but “did it still know when it was likely wrong?” — measuring calibration, abstention, and confidence–error coupling. *Deferred*.
- **Quantization × distribution shift.** Deployment reliability is closer to “retains under INT4 *and* shifted distribution” than to “retains under plain INT4.” Run the same matched pairs under compression-plus-shift as a second layer, after the clean quantization-only baseline. *Deferred*.
- **Format-comparison as its own axis.** Given the BF16 finding, comparing fragility *profiles across formats* (does the narrow/broad gap differ under FP8 vs. INT8 vs. INT4-weight-only?) is a natural extension once the single-format baseline exists. *Deferred*.

Metric

The test is **retention relative to each task’s own full-precision baseline**, not absolute accuracy. A hard task that starts at 60% has more room to fall than an easy one at 95%; comparing raw drops would compare apples to anvils.

For each task, at bit width w (here “width” means quantized bit depth — FP16, INT8, INT4, and so on):

$$R_w = \text{mean}(\text{score at width } w) / \text{mean}(\text{score at FP16})$$

Within each matched pair, at the same bit width and *under comparable scoring strictness*, the prediction is: $R_w(\text{narrow}) < R_w(\text{broad})$

(The “comparable scoring strictness” condition is load-bearing and is specified just below — it is the single most important guard against the experiment fooling itself.)

Chance-corrected retention. For task formats with a nonzero random, majority-class, or format-induced floor — binary decisions, multiple choice, constrained slots — report chance-corrected retention alongside the raw ratio:

$$R_w(\text{corrected}) = (\text{score at } w - \text{chance}) / (\text{score at FP16} - \text{chance})$$

and use the corrected value for the primary comparison whenever the chance floors differ across the two halves of a pair. Raw retention assumes the score floor is roughly zero; a constrained format whose collapse bottoms out at its guessing floor (e.g. 25% for 4-way choice) will look artificially stable under the raw ratio compared to a free-generation task that can fall to zero. Correcting to chance removes that distortion so the pair is compared on equal footing.

Report the paired difference, not just two curves. At each bit-depth rung, report $\Delta R = R_{\text{broad}} - R_{\text{narrow}}$ (and $\Delta R_{\text{adj}} = R_{\text{adj,broad}} - R_{\text{adj,narrow}}$ when chance floors differ), with bootstrap confidence intervals on the paired difference. This makes the experiment’s actual target a single quantity with its own uncertainty, rather than two separate curves a reader eyeballs into a gap. The prediction is $\Delta R > 0$ with an interval excluding zero; an interval that substantially overlaps zero is a flat result, not a weak signal. (Consistent with the small-pilot stance below: bootstrap intervals carry more interpretive weight here than p-values.)

Pair retention with a correctness check, because retention is blind to robust error.

Retention is (score under stress) / (score at baseline). If a task’s baseline answer is *already wrong* in a stable way — a common shortcut that happens to fail — then a retention of ~ 1.0 means the model perfectly retained a wrong answer, and the raw retention number will read as “robust” when it should read as a red flag. This is not a quirk; it is analytic, a structural limit of any retention-only metric. So each task in a pair carries, alongside its retention, an **adversarial correctness check**: at least one counterexample case where the plausible shortcut gives the wrong answer. Report retention *and* counterexample-survival together, and read them jointly: **high retention with a surviving correct answer is a genuine signal; high retention with a surviving wrong answer (the counterexample still failing under stress) is a robustly-wrong capability, not a robust one.** Without this column, the protocol can certify stable

wrongness as strength. The guard is cheap — one counterexample per task — and it closes the one hole that retention alone cannot see.

When a task is wrong, record **error identity, not just error presence** — is it the *same* wrong answer across stress rungs and paraphrases, or a *different/incoherent* wrong answer each time? This separates two failure modes that the bare wrong/right column conflates: a **stable** wrong answer is a learned shortcut or wrong rule (deployment-dangerous — it is exactly what benchmark-overfitting and reward-hacking produce, and it is the cell retention is blind to), while a **shifting or random** wrong answer is ordinary weak capability (less alarming, and not a robust shortcut). The full read is a 2×2: baseline-correctness × retention, with the wrong-and-retained cell split by error-identity. The cell to fear is *wrong at baseline, same wrong answer retained under stress* — stable, fluent, and invisible to any retention-only metric. Mislabeling random-wrong as robust-wrong would over-call the danger; the error-identity check is what keeps that distinction honest.

Metric symmetry is the central confound, and it must be controlled before the asymmetric version is ever defended. An earlier draft of this protocol scored broad tasks with a tolerant metric and narrow tasks with exact match, and called the asymmetry “part of the hypothesis.” That was wrong, and it is the single fastest way to fool this experiment. If the broad half is judged generously and the narrow half by exact match, then coarsening will *of course* show a larger retention drop on the narrow half — not because the capability is more fragile, but because exact match punishes any drift while a tolerant metric absorbs it. The experiment would then measure *scoring tolerance*, not *model fragility*, and the first run could look profound while being a grading artifact.

The governing principle:

The primary broad/narrow comparison must use **comparable scoring strictness**. Exact-match cliffs may be reported as *diagnostics*, but they cannot be the sole basis for claiming precision-demand fragility.

The fix is to make the **broad** task scorable under a strictness comparable to the narrow task — not by making it narrow, but by giving it a *canonical structured score*. “Summarize the passage” becomes “list the three required facts”; “explain the code” becomes “identify the main function calls and return types.” This can mean exact match where there is one valid output, but more often it means a **required key-fact checklist, normalized string match, constrained multiple-choice, JSON-field correctness, entity/state-slot accuracy, or a high-precision rubric with predefined acceptable variants**. The cost is real — a constrained broad task is slightly less “broad” — but for a first pilot that is the right trade: clean beats cinematic. The naturalistic, messy version is for Tier 1+.

The trap to avoid: do not constrain the broad task so hard that it becomes precision-demanding. If you do, you are comparing two narrow tasks and the absence of a gap proves nothing. The target is *comparable strictness, not identical narrowness* — the broad half should still tolerate equivalent phrasings within its checklist, while the narrow half flips on small deviations.

Scoring layers (score both halves under more than one lens). For each pair, compute retention under at least: a **strict/canonical** layer (checklist or exact-match) and a

tolerant/component layer (partial credit, slot accuracy, or a frozen rubric). The fragility signal counts only if the narrow half retains less than the broad half *under a comparable-strictness layer* — not only under exact match. If the gap appears under exact match and vanishes under partial credit, that is a metric-cliff artifact, not fragility.

Scorer-sensitivity control (the boring control that hides the knife). Add to the broad-broad and narrow-narrow negative controls a third: **same task, multiple scorers** (strict, tolerant, frozen-rubric) on a subset of outputs. Then ask: *does quantization change the disagreement between scorers?* If the strict scorer drops sharply and the tolerant one does not, you have found metric-sensitivity, not capability-fragility. If an LLM judge is used anywhere, freeze the judge model, prompt, rubric, temperature, and output format — otherwise the evaluator is a second moving model.

Guards. - Drop any pair scoring below ~0.20 at FP16 — retention ratios on near-floor tasks are noise. - Flag any broad task above ~0.98 at FP16 — apparent stability may be a ceiling effect rather than true robustness. - Report **raw score distributions** (histograms by bit depth), not only retention ratios — they reveal whether degradation is smooth, cliff-like, bimodal, or metric carnage wearing a lab badge. - Report **both** retention and raw scores. Retention is the test; raw scores are the audit trail that keeps it honest.

Sample size. Aim for 50–100 items per task, or the retention curves jitter and a real bend can't be told from sampling variance. With only four pairs, any significance test (e.g. Wilcoxon over the deltas) is a directional check at best — the plot and bootstrap intervals carry more interpretive weight than a p-value over four points. Four pairs is a pilot, not a verdict.

Optional, and worth it if tasks are step-structured: log the index of the first failing step. “Narrow tasks degrade first” is a result; “narrow tasks degrade first *and* the failure localizes to an early step that then cascades” is a stronger one, and consistent with existing step-localization findings in the quantization-and-reasoning literature.

The pre-declared decision rule

This is what makes it a measurement rather than a confirmation ritual. **Predeclare three outcomes, not two** — “confirmed / not confirmed” lets a metric artifact pass as confirmation; three outcomes force the result to survive being re-read as an artifact.

- **Outcome A — real fragility signal.** The narrow half retains less than the broad half *under a comparable-strictness scoring layer* (not only exact match), the scorer-sensitivity control shows the effect is not just metric-disagreement, and (if step-structured) first-error-step localization supports early collapse. → The fragility axis has measurable bite; expand it (more pairs, the salient-weight rescue test, then the provenance comparison).
- **Outcome B — metric-cliff artifact.** The narrow half collapses only under exact match, but partial-credit or comparable-strictness scoring shows retention similar to the broad half. → Scoring strictness manufactured the apparent gap. A useful warning, not a framework confirmation. Fix the scoring before interpreting anything.

- **Outcome C — pair / task confound.** The broad–broad or narrow–narrow negative controls show a retention gap, or FP16 baselines are mismatched. — Pair construction failed; redesign tasks before interpreting anything.

Only Outcome A counts as support, and only after the metric controls pass. If two or more contrast pairs reverse, or bootstrap intervals on the paired retention difference substantially overlap zero — **publish the flat result.** Do not rationalize it. A flat result means either the pair design needs work or the fragility axis needs a better formulation — and either is a real finding the framework should absorb.

The experiment must be able to come back “no.” If it can only confirm, it is worthless.

Where this sits in a larger program

This pilot is **Tier 0**. The structure below corrects an artifact of how it was assembled: the diagnostics were added one at a time, each as a serial “next tier,” but they are not actually a sequence — **they all reuse Tier 0’s inference runs**, so their marginal cost is *analysis*, not *compute*. The honest shape is one baseline plus a *fan* of parallel diagnostic tracks, not a chain.

Tier 0 — Baseline fragility probe (minimum viable). Test whether precision demand predicts retention under a specified quantization stress, with comparable scoring and the three-outcome decision rule. Ship the one-page checklist with it. **Two model sizes (e.g. an 8B and a 70B-class) are strongly recommended** — same task suite, twice the signal, no redesign later — *but a single open-weights model is a legitimate Tier 0*, and a second size is the first expansion if compute is the binding constraint. The point of Tier 0 is that it stays runnable by someone without a large cluster; do not let “two sizes would be better” raise the entry barrier past “runnable at all.” Output: paired retention curves, scale comparison if two sizes, three-outcome decision.

Tier 1 — Parallel diagnostic tracks (all reuse Tier 0’s runs). Run only if Tier 0 returns Outcome A — a real fragility signal. If Tier 0 returns Outcome B (metric-cliff artifact) or Outcome C (pair/task confound), there is no signal to diagnose, and running these tracks would be diagnosing a ghost; fix Tier 0 first. Given Outcome A, these are low marginal cost — the inference is already done; what’s added is analysis. Each maps to a logged implication and carries its own pre-declared decision. They are *named and gated here, not internally over-specified* — exact metrics and thresholds belong in whichever track is pursued *after Tier 0 produces a signal worth analyzing*, not before.

- **1a — Salient-weight rescue (structural localization).** *Run only if Tier 0 returns Outcome A.* Tests B13: is fragility about fine spacing or outlier channels? Protect top-*k* salient weights/channels, rerun narrow tasks at the same bit depth. If rescue recovers narrow more than broad — fragility is channel-local (fix architecture/compiler); if not — fine-spacing (fix training).
- **1b — Uncertainty / calibration retention (silent failure).** *Run only if Tier 0 returns Outcome A.* Tests B20, the framework’s core worry, on outputs already generated: does narrow lose *calibration* more than broad under the same stress (ECE, confidence–error coupling, abstention, overconfidence-on-wrong)? A deployment-relevant fragility signal

even when accuracy looks stable. Highest leverage, lowest marginal cost — which is why it belongs first among the parallel tracks, not deferred.

- **1c — Cross-stress correlation (generalizability).** *Run only if Tier 0 returns Outcome A.* Tests B28: is this about INT4 or about precision demand? Rerun the same pairs under a second stress at matched performance cost (e.g. ~50% unstructured pruning, or activation noise); correlate task-level retention *ranks* across stresses. High rank-correlation → capability fragility, not a quantization artifact. This is the track that upgrades the probe from “quantization-sensitivity test” to “general fragility probe.”
- **1d — Prompting-recovery diagnostic (structural vs. implementation).** *Run only if Tier 0 returns Outcome A.* Tests B27: can inference-time scaffolding fix the drop? Rerun failed narrow tasks with chain-of-thought or tool use under the same stress. Recovers — implementation fragility (promptable); doesn’t — structural fragility (requires training, not prompting).

Tier 2 — Provenance comparison. Compare code-heavy vs. text-heavy (or world-model) variants, now that a within-model baseline exists. This is where provenance × fragility finally becomes testable. (Substantial controlled prior art already exists — see literature notes — so this is positioned as confirmation/extension, not discovery.)

Tier 3 — Targeted structure block. Add a small, high-checkability intervention block — training examples whose structure exercises the fragile distinction directly — then rerun, to ask whether the intervention *widens a narrow ridge* (improves retention under coarsening) rather than merely raising raw accuracy. Stated without any appeal to “pressure”: under fixed architecture, algorithm, quantization protocol, and scoring, a small targeted structure block should improve retention on the fragile task more than an equal-token block of broad, undifferentiated data.

Quick reference:

Tier	Focus	Cost	Dependency
0	Baseline retention (two sizes if feasible)	Medium	None
1a	Salient-weight rescue	Low	Tier 0 outputs
1b	Uncertainty / calibration retention	Very low	Tier 0 outputs
1c	Cross-stress correlation	Medium	Tier 0 design
1d	Prompting recovery	Low	Tier 0 failures
2	Provenance comparison	High	Tier 0 baseline
3	Targeted	High	Tier 1 mechanism

Tier	Focus	Cost	Dependency
	structure block		result

Nothing past Tier 0 is promised, and **every track is contingent on Tier 0 producing a signal** — if Tier 0 returns flat (Outcome B or C), there is no effect to localize, and the fan does not open. The strategic summary — “passing at FP16 is not qualification” — is what *emerges after 1b and 1c*, if they hold; it is not a separate workstream and not a claim the design assumes. Tier 0 is the door.

Where the probe is actually looking

The probe does not map the mountain. It load-tests selected channels.

Return briefly to the analogy — weights as mountain, activations as water, training as carving — and the point becomes clear. We are not asking how a channel was formed. We are not testing provenance. We are not proving that wide basins or narrow ridges exist in parameter space. We are asking one local question about a frozen structure: how much numerical coarsening can this behavior survive?

In mechanistic terms, we apply a specified stress — format, method, calibration set and hash, bit-depth ladder — to a fixed model, then measure retention:

$$R_w = \text{score under stress} / \text{score at full precision}$$

A broad task, such as structured summarization, has many acceptable routes to correctness. Several nearby outputs still count. In the analogy this is a floodplain — the water can shift and still reach an acceptable delta.

A narrow task, such as exact temporal tracking, deterministic code execution, multi-step arithmetic, variable binding, or a tool-use permission boundary, has few acceptable routes. A small deviation flips correctness. In the analogy this is a slot canyon: a small change in the channel or carried state sends the flow to the wrong outlet.

If the narrow task retains less than its matched broad task under comparable scoring strictness, and the negative controls pass, we have evidence of a capability-level fragility signature. The narrower channel failed earlier under coarsening — mechanistically, the behavior depended on distinctions the stressed representation preserved less reliably.

Tier 0 does not test whether the river-and-canyon picture is true. It tests whether precision-demanding behaviors lose more retained performance than matched lower-precision-demand behaviors under a specified stress.

Tier 1 asks what *kind* of fragility was exposed. Salient-weight rescue tests whether the failure is concentrated in a few critical numerical structures. Uncertainty retention tests whether the model still knows when it is wrong. Cross-stress correlation tests whether the same behavior fails under different stresses, pointing to general fragility rather than a quantization artifact. Prompting recovery tests whether inference-time scaffolding can rescue the behavior or whether weight-level repair is required.

The probe is not a map. It is a controlled load test on specific paths. Retention is the behavioral signal we can measure; local geometry is the interpretation we may cautiously draw. The method is useful because the measurement stands even if the analogy is set aside.

What a positive result would and would not mean

It **would** mean: precision demand predicts quantization sensitivity in a measurable, repeatable way — fragility is measurable under this probe, not merely descriptive. And if the directional signal holds, the retention curve becomes a yardstick other labs can adopt *without adopting the whole worldview* — the measurement stands on its own, and someone can run it, report it, and build on it while ignoring the river-and-canyon framing entirely. That portability is a feature: the protocol asks to be tested, not believed.

The honest positioning, given what the literature already shows: compression effects are format-dependent, task-dependent, calibration-sensitive, locally diagnosable, sometimes repairable with targeted data, and not fully captured by accuracy alone. So the contribution here is **not discovering fragility — the field has established that compression hurts precision-demanding capabilities. It is a clean, controlled way to isolate, measure, and qualify it**: matched pairs that hold everything but precision demand constant, comparable scoring strictness so the metric can't manufacture the gap, format and calibration treated as part of the stress instrument rather than as setup noise, and an honest set of outcomes including the one where the apparent effect dissolves. That narrows the novelty claim and strengthens the method claim — which is the right trade. The probe is extensible to the deferred dimensions above (uncertainty retention, distribution shift, cross-format comparison) once the clean single-format baseline exists.

One limit is structural and worth stating plainly, because it bounds what any retention result can claim: **retention measures survival, not correctness**. A capability that is stably *wrong* — a shortcut that gives the wrong answer but does so robustly — retains under stress just as well as a correct one, and a retention-only metric will score it as strong. This is why the correctness-check column above is not optional: without it, the instrument is blind to robust error, and “high retention” can certify exactly the failure (benchmark-overfit shortcuts, reward-hacked behavior) it should be flagging. The probe measures how well a behavior *survives* coarsening; it does not, on its own, measure whether that behavior was *right*. Pairing every retention number with an adversarial correctness check is what keeps a robust wrong answer from being read as a win.

It **would not** mean: that the margin-geometry interpretation is confirmed, that provenance predicts fragility, or that the analogy in the parent papers is mechanically true. Those are separate questions requiring separate work. This tests one prediction. That is all it claims to do.

Expansion notes: scale, adoption, and deployment-relevant failure

Three notes on where this goes, kept short on purpose — Tier 0 stays one clean model and four-plus-controls pairs; these name the path without turning the pilot into a Christmas tree.

Scale. The initial pilot may use one model, but generalization requires repeating the identical suite across **at least two model sizes**. A8 ultimately asks whether retention predicts *deployment reliability*, and a result from a single size is a starting point, not a claim — retention

behavior may itself scale, and a one-model finding cannot tell you whether it does. Plan the second size in from the beginning so the result is not silently size-specific.

Adoption artifact. A method trapped in prose does not travel. Every release of this protocol should ship the one-page checklist below (and, ideally, reference task templates), so someone can run it without reverse-engineering the design from paragraphs. Boring is how methods travel.

A safety-relevant narrow pair (exploratory). Include one narrow task where the failure mode is exactly the one the framework worries about — *fluent surface behavior preserved while a fine-grained constraint fails*: a tool-use permission boundary (“do not call tool X unless condition Y holds”), a state-dependent refusal (allowed in one context, disallowed after a condition changes), or a forbidden action embedded among allowed ones. **This is labeled exploratory and it is not a claim about alignment robustness.** Its purpose is narrow: to test whether the metric-symmetry design works on a task where *silent failure actually matters* — not to demonstrate that safety degrades under compression. The deployment relevance is in the task type; the claim remains only “the probe behaves as designed here.” Reading this as evidence of alignment fragility would be exactly the overclaim the rest of this protocol exists to prevent.

One-page run checklist

The minimum-viable artifact. Declare each item before running; report each alongside results.

- **Model name(s) / size(s)** — list all runs; note whether this is a single-size pilot or a two-size Tier 0
 - **Quantization method and format** (e.g. GPTQ W4A16, SmoothQuant W8A8, FP8 W8A8)
 - **Weights-only, or weights + activations**
 - **Calibration file hash** (identical file at every bit depth)
 - **Calibration distribution summary** (domain mix, token/length/language distribution; resemblance to evaluated tasks)
 - **Bit-depth ladder** (e.g. FP16 → INT8 → INT4)
 - **Matched-pair construction** (same source/length/skeleton; split by precision demand, not complexity; exclusions recorded)
 - **Comparable scoring strictness** (canonical structured score on the broad half; not tolerant-vs-exact-match)
 - **Negative controls** (broad–broad and narrow–narrow; must show no within-pair gap)
 - **Scorer-sensitivity control** (same task, multiple scorers; does quantization change their disagreement?)
 - **Raw score distributions** (histograms by bit depth, not only retention ratios)
 - **Retention ratio** (R_w relative to each task’s own FP16 baseline)
 - **First-error-step logging** (where tasks are step-structured)
 - **Declared outcome** (real signal / metric-cliff artifact / pair confound)
-

Companion to The River and the Canyon and What Kind of Water Carves the Mountain?
Proposed, not performed. If you have the compute and want to run it — or want to break the
design — that is exactly the point of writing it down. Written by E. A. Flores, Apiana AI,
Inc. Licensed CC BY-NC 4.0.